🏫 自習室入退室管理システム：テクニカル・マスター・ガイド (詳細仕様書)

本ドキュメントは、システム開発者、技術保守担当者、または次世代の技術リーダーに向けた詳細な技術マニュアルです。システムの全挙動の把握、環境の再構築、保守、拡張に必要な情報を網羅しています。

---

1. アーキテクチャ再定義 (The 4-Layer Architecture)

本システムは、高い可用性とセキュリティを両立させるため、以下の4つの層で構成されています。

1.1 Infrastructure Layer (Docker / Ubuntu VPS)

- **役割**: アプリケーションを「環境に依存せず」安定稼働させる基盤。
- **詳細**: `docker-compose.yml` により、Next.js アプリケーションが独立したコンテナとして実行されます。これにより、サーバーのOS（Ubuntu等）を直接汚すことなく、同一サーバー上の別サイト（WordPress等）とのポート競合を防ぎます。
- **Network**: Nginx が 80/443 ポートで待ち受け、 `proxy_pass` を通じてコンテナ内の `3000` ポートへとトラフィックをブリッジします。

1.2 Application Layer (Next.js 14 App Router)

- **役割**: ビジネスロジックの実行、ユーザー認証、UI の提供。
- **詳細**: フロントエンド（React）とバックエンド（API Routes）がシームレスに結合したモノリス構成です。
- **認証**: NextAuth.js を中心に据え、Google OAuth 2.0 によるログイン、 `middleware (proxy.ts)` による認可ゲートウェイ、そして `jsonwebtoken` による QR コードの動的署名を組み合わせています。

1.3 Database Layer (Supabase / PostgreSQL)

- **役割**: 永続データの保持とトランザクション整合性の保証。
- **詳細**: Prisma ORM を介して Supabase 上の PostgreSQL と通信します。単純な CRUD 操作だけでなく、 `$transaction` を用いて「入室処理」と「ログ生成」をアトミックに実行し、データの不整合を物理的に排除しています。

1.4 External Services Layer (Google Cloud & Gmail)

- **役割**: 認証基盤、および非同期での通知・承認機能。
- **詳細**: Google Cloud Console による OAuth 設定と、Gmail SMTP (Nodemailer) による保護者への即時メール通知を担います。

---

2. API 詳細仕様 (Ultimate Deep Dive)

2.1 QRコード発行 (`/api/qr`)

- **フロー**:
  1. `getServerSession` でログイン済みユーザーを特定。
  2. 環境変数 `NEXTAUTH_SECRET` が存在するか厳格にチェック。
  3. ペイロード（ `userId` , `studentId` , `timestamp` , `purpose: "qr"` ）を作成。
  4. **署名**: `jwt.sign` で 60秒間有効なトークンを生成。
- **意図**: 有効期限を短く（60秒）設定し、フロントエンドで 30秒ごとに更新させることで、万が一 QR が流出してもその効力を数分で失わせる設計です。

2.2 スキャン検証 (`/api/scanner/verify`)

- **フロー**:
  1. トークンの書式（JWTかつ2048文字以内）をバリデーション。
  2. `jwt.verify` で署名の正当性と有効期限を一気にチェック。
  3. `decoded.purpose === "qr"` であることを確認（他の用途のJWTでの不正ログインを防止）。
  4. **二重読み込み防止**: `AttendanceLog` の最新レコードを検索し、前回の打刻から **1分以内** であればエラー（429 Too Many Requests）を返します。
  5. 保護者による有効期限（ `validUntil` ）内であるかをチェック。

2.3 入退室処理 (`/api/checkin`)

- **フロー**:
  1. スキャン検証（上記 2.2 と同様）を実行。
  2. **トランザクション開始**:
     - `User.update` : 状態（ `IN` / `OUT` ）を反転し、座席番号をセット/クリア。
     - `AttendanceLog.create` : アクション内容を記録。
  3. **非同期通知**: `sendNotificationEmail` を `await` せずに呼び出します。これにより、メールサーバーの遅延が受付画面のレスポンス速度に影響を与えません。

---

3. データベース「完全」定義書 (Entity Details)

Prisma Schema (`schema.prisma`) に基づく、各フィールドの役割と設計意図です。

3.1 `User` テーブル (核となる存在)

- `studentId` : Googleメールのローカルパート（ `@` の前）を抽出した文字列。ユニーク制約。
- `currentStatus` : `"IN"` または `"OUT"` 。初期値は `"OUT"` 。
- `isRegistered` : 初回ログイン時に学籍番号と保護者メールを登録したかの証跡。
- `validFrom` / `validUntil` : 保護者承認による利用可能期間。これを過ぎるとログインはできても QR 発行がロックされます。
- `currentSeat` : 在室中の座席番号。 `OUT` 時は必ず `null` に設定されます。

3.2 `AttendanceLog` テーブル (履歴)

- `action` : `"IN"` または `"OUT"` 。
- `timestamp` : 打刻されたサーバー時刻。デフォルトは `now()` 。
- `remarks` : 「深夜の自動退出」や「管理者による強制ログアウト」などの特殊なケースで理由を記述します。

3.3 `SystemSetting` テーブル (動的設定)

- `key` : 設定の名前（例: `"seat_layout"` , `"used_permission_[Hash]"` ）。
- `value` : JSON 文字列。変更時にデータベースを直接触る必要をなくし、管理画面から変更可能にしています。

---

4. フロントエンド・ステート管理と UI ロジック

4.1 ダッシュボード (`app/page.tsx`)

- **30秒ループ**: `useEffect` 内で `setInterval` を回し、30秒ごとに `/api/qr` から最新のトークンと、 `/api/seats` から最新の混雑状況を取得します。
- **スクショ防止アニメーション**:
  - **枠線アニメーション**: CSS の `conic-gradient` を利用した虹色の枠線が回転し続けます。これにより、静止画（スクショ）ではないことを管理者が一目で判別できます。

- **リアルタイム時計**: 秒単位で刻む時計が表示されます。

4.2 座席表 (`components/SeatMap.tsx`)

- 座席選択時に、既に他の誰かが座っている座席を視覚的に（グレーアウト等で）無効化します。
- レイアウトは `SystemSetting` から取得した二次元配列に基づき、動的にレンダリングされます。

---

5. セキュリティ設計 (The Threat Model)

本システムは以下の脅威に対して対策を講じています。

脅威	対策 (Mitigation)
ドメイン外のログイン	`NextAuth` の `signIn` コールバックで `@niigata-meikun.ed.jp` 以外を物理的に遮断。
QRコードの偽造	`HS256` 署名付き JWT。秘密鍵はサーバー上の環境変数のみに存在。
QRコードの流出・使い回し	有効期限 60秒設定 ＋ 受付側での 1分間連続打刻防止。
管理者画面への不正アクセス	Next.js 16 の `proxy.ts` (Edge Middleware) でメールアドレスをホワイトリスト形式で検証。
保護者承認リンクの再利用	使用済みリンクのシグネチャ・ハッシュを DB に記録し、マジックリンクを使い切り化。

---

6. 運用・保守・トラブルシューティング (Expert Level)

6.1 Prisma による DB メンテナンス

- **スキーマ変更**: `schema.prisma` を書き換えた後、以下のコマンドを実行します。

```
sudo docker compose exec app npx prisma db push
```

※ `db push` は本番環境での迅速な同期に向いていますが、大規模な破壊的変更時は `migrate dev` (マイグレーション) の利用を検討してください。

6.2 Docker コンテナの健康診断

- **アプリが立ち上がらないときは**:

```
sudo docker compose logs -f app
```

  - `ENOTFOUND` : データベースの URL または Gmail 接続設定が間違っています。
  - `Port 3000 already in use` : 他のプロセスがポートを占有しています。

6.3 Gmail の送信制限 (Quota)

- Gmail (無料版) には 1日 500通程度の送信制限があります。利用者が急増し、1日 250人（入退室それぞれ 1通ずつ）を超え始めた場合は、Google Workspace (有料版) または SendGrid 等の外部メール配信サービスの導入を検討してください。

---

8. 保守担当者向け：用語解説 (Technical Glossary)

技術に詳しくない方でも、文脈を理解できるようにするための補足です。

用語	読み方	かみ砕いた解説
アーキテクチャ	Architecture	システム全体の「構造」や「設計図」のこと。どの部品がどう繋がっているかを示します。
インフラ	Infrastructure	サーバーやネットワークなど、システムを動かすための「土台」のこと。
Docker	ドッカー	アプリを「コンテナ（箱）」に詰めて持ち運べるようにする技術。サーバーの環境を汚さずに済みます。
データベース	DB	ユーザー情報やログを整理して保管しておく「倉庫」。今回は PostgreSQL (Supabase) を使用。
ORM (Prisma)	プリズマ	プログラムからデータベースを簡単に操作するための「翻訳者」。
Next.js	ネクスト	フロントエンド（画面）とバックエンド（機能）を同時に作れる、現代の強力な道具（フレームワーク）。
API	エーピーアイ	異なるプログラム同士が情報をやり取りするための「窓口」。
JWT	ジェイダブリューティー	情報を暗号化して伝える「署名付きのチケット」。QRコードの正体です。
GitHub / リポジトリ	ギットハブ	プログラムのソースコードを保管・共有・履歴管理するための「クラウド倉庫」。
環境変数 (.env)	ドットエンブ	パスワードや設定など、ソースコードに直接書きたくない「秘密の情報」を置く場所。
トランザクション	Transaction	「いくつかの処理をまとめて、1つでも失敗したら全部なかったことにする」という安全策。

---

9. メンテナンス・チートシート (トラブル逆引き実例集)

ソースコードの更新から、予期せぬエラーの解決まで、現場で発生しうる 10 のケースに対する「原因」「手順」「予防策」をまとめました。

【ケース A】デザイン・文言などの小さな修正

- **原因**: 画面上の誤字脱字や、配置などの軽微なプログラム変更。
- **手順**:
  1. SSH で本番サーバーにログイン。
  2. `cd /var/www/attendance-system` でプロジェクトフォルダへ。
  3. `git pull origin main` で最新コードを GitHub から取得。
  4. `sudo docker compose up -d --build` で再構築して起動。
- **予防**: 常に `main` ブランチを最新に保ち、自分以外のメンバーが勝手に書き換えていないか GitHub の履歴を確認しましょう。

【ケース B】学籍番号など、記録する情報を増やした時

- **原因**: `prisma/schema.prisma` (データベースの設計図) を変更した。
- **手順**:
  1. 上記【ケース A】の手順 1〜3 を実行。
  2. `sudo docker compose exec app npx prisma db push` を実行（DBの形を更新）。
  3. `sudo docker compose up -d --build` で再起動。
- **予防**: DBの変更はやり直しが難しいため、実行前に Supabase のダッシュボードでバックアップを推奨します。

【ケース C】パスワードやメール設定 (.env) を変えた時

- **原因**: 設定ファイル内の Gmail パスワードや Google クライアント ID などの更新。
- **手順**:
  1. `nano .env` でファイルを編集し、値を書き換えて保存 (`Ctrl+O` → `Enter` → `Ctrl+X`)。
  2. `sudo docker compose restart app` でアプリのみ再起動。
- **予防**: `.env` を誤って消さないよう、バックアップ版 (`.env.backup`) を作っておくと安全です。

【ケース D】原因不明の動作不良 (フリーズ等)

- **原因**: メモリの使いすぎ (Memory Leak) や、コンテナ内のゴミファイル。
- **手順**:
  1. `sudo docker compose down` で一旦すべての箱（コンテナ）を廃棄。
  2. `sudo docker compose up -d --build` で最初から組み立て直して起動。
- **予防**: 夜間の自動再起動 (Cron) スケジュールを VPS 側に追加することを検討してください。

【ケース E】データベース接続エラー ("PrismaClientInitializationError")

- **原因**: Supabase 側のパスワード変更や、プランの停止（非アクティブによる休止）。
- **手順**:
  1. Supabase ダッシュボードでプロジェクトが「Active」か確認。
  2. `.env` の `DATABASE_URL` が正しいか確認。
  3. サーバーから `ping` 等で外への通信が通じているか確認。
- **予防**: Supabase は 1週間アクセスがないと一時停止するため、定期的に管理画面などを開くようにしてください。

【ケース F】Google ログインができない ("OAuth Error")

- **原因**: Google Cloud Console での「承認済みのリダイレクト URI」の不一致、クライアントシークレットの無効化。
- **手順**:
  1. Google Cloud Console でクライアント ID の設定を確認。
  2. 承認済みのリダイレクト URI が `https://YOUR_DOMAIN/api/auth/callback/google` になっているか確認。
  3. `NEXTAUTH_URL` が `.env` で正しくドメイン名 (`https://...`) になっているか確認。
- **予防**: Google 側の設定を変更したら、反映まで数分〜数時間かかる場合があります。

【ケース G】メールが届かない

- **原因**: Gmail 1日の送信上限 (Quota) 到達、または「アプリパスワード」の無効化。
- **手順**:
  1. Gmail の管理アカウントにログインし、送信済みトレイを確認。
  2. Google アカウントのセキュリティ設定で「アプリパスワード」を再生成。
  3. 生成したパスワードを `.env` の `EMAIL_PASS` に反映し、【ケース C】の手順を実行。

- **予防**: 利用者が増えた場合は SendGrid などの専用メール配信サービスの導入を検討してください。

【ケース H】サーバーが極端に重い、反応しない

- **原因**: Docker ログの肥大化、または不要な古いコンテナイメージの堆積（ディスク不足）。
- **手順**:
  1. `df -h` でディスク容量を確認。
  2. `sudo docker system prune -a` で不要なキャッシュやイメージを一括削除。
  3. `sudo docker compose logs -f app` でエラーの洪水が起きていないか確認。
- **予防**: 定期的に `docker system prune` を手動、または自動で実行する習慣をつけましょう。

【ケース I】生徒の QR コードが常に「期限切れ」になる

- **原因**: サーバーの時刻が実際の時刻と大きくズレている。
- **手順**:
  1. `date` コマンドでサーバー内部の時刻を確認。
  2. `sudo timedatectl set-ntp on` でネットワーク経由の時刻同期を有効化。
  3. `timedatectl` で Timezone が `Asia/Tokyo` になっているか確認。
- **予防**: VPS の初期設定時に NTP (時刻同期) の設定は必須項目です。

【ケース J】サイトに大きな「⚠」マークが出てアクセスできない

- **原因**: SSL 証明書 (Let's Encrypt) の期限切れ、または Certbot の自動更新失敗。
- **手順**:
  1. `sudo certbot certificates` で証明書の残り日数を確認。
  2. `sudo certbot renew` で更新を強制実行。
  3. `sudo systemctl reload nginx` で設定を再読み込み。
- **予防**: 通常は自動で更新されますが、Nginx の設定を変更して構文エラーがあると自動更新も止まります。

---

10. 将来の拡張へのロードマップ (Legacy)

このシステムは「美しく管理しやすい」状態を維持していますが、さらなる進化の余地があります。

1. **統計機能の視覚化**: 月間の「自習王」をランキング表示したり、個人の学習時間の推移を Chart.js でグラフ化する。
2. **混雑予測 AI**: 過去の履歴を元に「今日の放課後の混雑具合」を予測表示する。
3. **複数施設対応**: `SystemSetting` を拡張し「第2自習室」「パソコン実習室」など施設ごとのレイアウトを切り替え可能にする。
4. **技術負債の定期解消**: Next.js, Prisma, Node.js のバージョンを年に一度はアップデートし、セキュリティホールを塞ぐ。

> *このシステムには、新潟明訓高校パソコン部の知恵と情熱が詰まっています。 2026年4月5日*